

Competitive Programming

Section 34: Probability and Expected Value

Competitive Programming Course

2026-07-06

Table of contents

1	Expected value in programming contests	3
2	Linearity of expectation	4
3	Geometric waiting time	5
4	Coupon collector	6
5	Modular harmonic sums	7
6	Worked expectation table	8
7	State-based expectation	9
8	Common mistakes	10
9	Submission checklist	11

1 Expected value in programming contests

Expected value is the long-run average value of a random variable. If a value x occurs with probability p , it contributes $p \cdot x$ to the expectation.

When a problem asks for expected value modulo a prime, represent a rational number:

```
a / b
```

as:

```
a * inverse(b) mod MOD
```

This uses modular inverses from Section 29.

2 Linearity of expectation

The most important rule:

$$E[X + Y] = E[X] + E[Y]$$

This is true even if X and Y are dependent.

For independent bonus events, event i contributes:

```
v[i] with probability p[i]/q[i]
0    otherwise
```

So:

```
E[event i] = v[i] * p[i] / q[i]
E[total] = sum_i E[event i]
```

3 Geometric waiting time

If each attempt succeeds with probability p , the expected number of attempts until the first success is:

$$1 / p$$

Derivation:

$$\begin{aligned} E &= 1 + (1-p)E \\ pE &= 1 \\ E &= 1/p \end{aligned}$$

If $p = a/b$, then:

$$E = b/a$$

4 Coupon collector

There are n coupon types. Suppose you already have i distinct types. The probability that the next draw gives a new type is:

$$(n-i)/n$$

The expected waiting time for the next new type is:

$$n/(n-i)$$

Add stages for $i = 0 \dots n-1$:

$$\begin{aligned} & n/n + n/(n-1) + \dots + n/1 \\ &= n * (1 + 1/2 + \dots + 1/n) \end{aligned}$$

5 Modular harmonic sums

For many coupon-collector queries, precompute:

$$H[i] = 1 + 1/2 + \dots + 1/i \bmod \text{MOD}$$

Then:

$$\text{answer}(n) = n * H[n] \bmod \text{MOD}$$

6 Worked expectation table

Suppose a task has three independent bonuses:

bonus	value	probability
10		$1/2$
6		$1/3$
9		$2/3$

The expected total is not the most likely total. It is the sum of each bonus's contribution:

$$\begin{aligned} &10 * 1/2 + 6 * 1/3 + 9 * 2/3 \\ &= 5 + 2 + 6 \\ &= 13 \end{aligned}$$

Linearity lets us avoid enumerating all 2^3 subsets of bonuses. This is the main contest trick: define indicator variables for events and sum their expectations.

7 State-based expectation

Some problems define expectation recursively. If $E[\text{state}]$ is the expected remaining cost, write one equation per state:

```

$$E[\text{state}] = \text{immediate\_cost} + \sum \text{probability}(\text{next}) * E[\text{next}]$$

```

If transitions only go to “smaller” states, compute by DP. If transitions can return to the same state, isolate the variable algebraically, like the geometric waiting-time derivation.

8 Common mistakes

Using floating-point output for a modular problem.
Assuming linearity requires independence.
For geometric waiting time, outputting p instead of $1/p$.
For coupon collector, using n/i with i in the wrong direction.
Forgetting to convert denominators with modular inverses.

9 Submission checklist

1. Is the problem asking for exact modular expectation or decimal output?
2. What random variables can be summed?
3. Can the process be split into stages?
4. Are denominators nonzero modulo MOD?
5. Are all subtractions and fractions normalized?